

[CS209A] Practice 1 - JVM

Deadline: 2 weeks from now. Should be checked onsite (during labs).

Task 1

1. Compilation and Execution:

- Skip `javac` and run `java Example1` directly. Observe the error message.
- compile and run the program correctly.

2. Class Loading Observation:

- Compile the program: `javac Example1.java`.
- Run the program with the following command to see the class loading process:

```
java -verbose:class Example1 > class_loading_log.txt
```

> `class_loading_log.txt` saves the output to a file.

3. Log Analysis:

- Open the `class_loading_log.txt` file.
- **Analyze Manually:** Can you find the point where your `Example1` class is loaded? What classes were loaded by the JVM *before* it could load your `Example1` class?
- **Use AI Assistant:** Feed the log to ChatGPT/DeepSeek and ask: "*Can you analyze this JVM class loading log and explain the sequence and purpose of the main groups of classes that are loaded?*"
- **Reflect:** Compare your findings with the AI's explanation. Why does the JVM need to load so many classes just to run a simple program?

Task 2

1. Discovering the Recursive Call:

- Use `javap` to disassemble the bytecode. (Make sure `Example1.java` is compiled.)
- **Find and Record:** Look at the bytecode for the `fib` method. Can you identify the set of instructions that corresponds to the recursive call `return fib(n - 1) + fib(n - 2);`? (Hint: Look for `invokestatic` (to call the method itself), `isub` (for subtraction), and `iadd` (for addition).)

2. Modify and Compare:

- Modify `Example1.java` to add a method `static int add(int a, int b)`.
- Use `javap` to inspect the new bytecode.

Task 3

1. Discovering the GC events

- Run with `java -XX:+PrintGCDetails Example2` and `java -Xmx64m -XX:+PrintGCDetails Example2`, then compare the number and type of GC events (look for `Pause Young` and `Pause Full` in the logs).

2. Trigger an `OutOfMemoryError`

- Modify loop count and array size to observe changes in the number and duration of GC events.
- Determine the threshold values for the loop count and array size that cause an `OutOfMemoryError` to occur.
- Reflect: How could you avoid memory overflow by changing code or JVM parameters?

Evaluation

The practice will be checked by teachers or SAs. What will be tested:

1. That you captured the `-verbose:class` log of `Example1` and used an AI tool for analysis.
2. That you used `javap -c` to find the bytecode of `Example1` for the recursive call in the `fib` method.
3. That you modified the code (added `add` method), recompiled, and used `javap` to inspect the new bytecode.
4. That you ran `Example2` with and without `-Xmx64m`, observing the difference in GC behavior.
5. That you attempted to trigger an `OutOfMemoryError` by modifying the code and/or JVM parameters.

Late submissions after the deadline will incur a 20% penalty, meaning that you can only get 80% of this practice's score.